LAPORAN UTS
SISTEM PARALEL DAN TERDISTRIBUSI

PUB-SUB LOG AGGREGATOR DENGAN IDEMPOTENT CONSUMER DAN DEDUPLICATION

Mata Kuliah : Sistem Paralel dan Terdistribusi
Kelas : B

Nama : Riza Luthfie Rusmana
NIM : 11231088

PROGRAM STUDI INFORMATIKA
INSTITUT TEKNOLOGI KALIMANTAN
2026

DAFTAR ISI

DAFTAR ISI **2**
BAB 1 RINGKASAN SISTEM DAN ARSITEKTUR **3**
1.1. Deskripsi Sistem 3
1.2. Diagram Arsitektur 3
1.3. Komponen Utama 4
1.4. Endpoint API 4
BAB 2 BAGIAN TEORI **5**
2.1. T1: Karakteristik Utama Sistem Terdistribusi dan Trade-off Pub-Sub Log Aggregator (Bab 1) 5
2.2. T2: Perbandingan Arsitektur Client-Server vs Publish-Subscribe untuk Aggregator (Bab 2) 5
2.3. T3: At-Least-Once vs Exactly-Once Delivery Semantics (Bab 3) 6
2.4. T4: Skema Penamaan untuk Topic dan Event ID (Bab 6) 6
2.5. T5: Ordering pada Sistem Terdistribusi (Bab 5) 7
2.6. T6: Failure Modes dan Strategi Mitigasi (Bab 8) 7
2.7. T7: Eventual Consistency melalui Idempotency dan Deduplication (Bab 7) 8
2.8. T8: Metrik Evaluasi Sistem (Bab 1 hingga Bab 7) 8
BAB 3 KEPUTUSAN DESAIN IMPLEMENTASI **10**
3.1. Idempotency 10
3.2. Deduplication Store 10
3.3. Pipeline Asinkron 10
3.4. Ordering 10
3.5. Docker dan Containerization 11
BAB 4 ANALISIS PERFORMA DAN METRIK **12**
4.1. Hasil Unit Tests 12
4.2. Hasil Stress Test 13
4.3. Hasil Docker Compose 13
4.4. Hasil Pengujian Endpoint API 15
REFERENSI **17**

BAB 1

RINGKASAN SISTEM DAN ARSITEKTUR

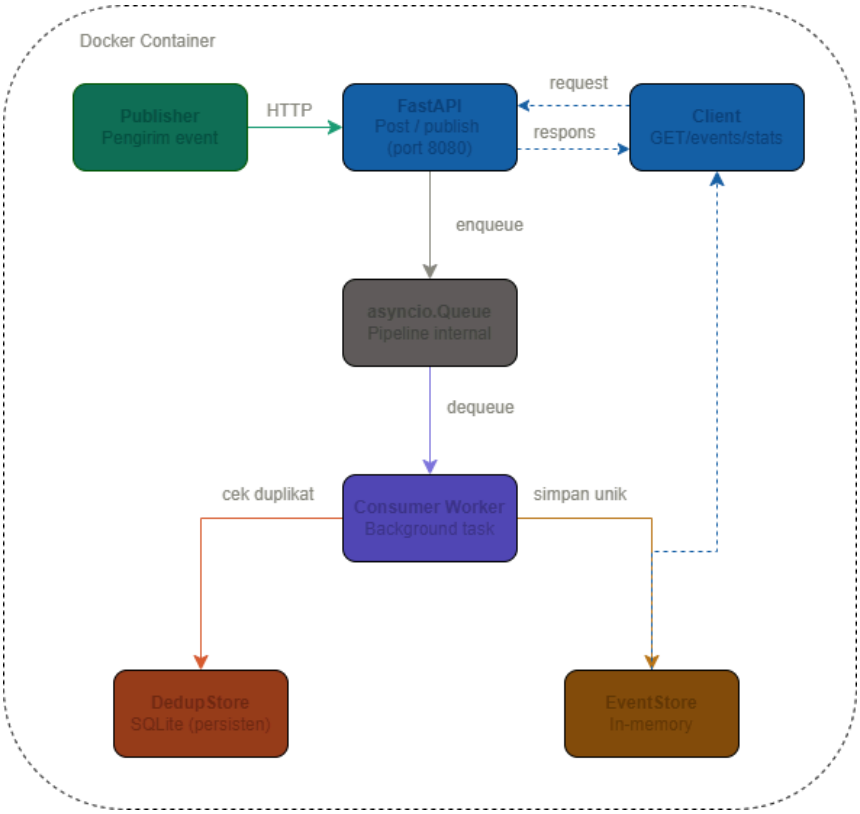
1.1. Deskripsi Sistem

Sistem yang dibangun adalah sebuah Pub-Sub Log Aggregator berbasis Python menggunakan framework FastAPI. Sistem ini menerima event atau log dari publisher melalui HTTP, memprosesnya secara asinkron melalui consumer yang bersifat idempoten, dan memastikan tidak ada event duplikat yang diproses lebih dari satu kali melalui mekanisme deduplication berbasis SQLite.

Sistem berjalan sepenuhnya di dalam Docker container dan tidak bergantung pada layanan eksternal apapun. Seluruh komunikasi jaringan bersifat lokal di dalam container.

1.2. Diagram Arsitektur

Berikut adalah diagram arsitektur sistem Pub-Sub Log Aggregator yang dibangun dalam tugas ini.



Gambar 1.1 Diagram Arsitektur Pub-Sub Log Aggregator.

Berdasarkan Gambar 1.1, sistem terdiri dari empat komponen utama yang saling terhubung. Publisher mengirimkan event melalui HTTP ke endpoint POST /publish pada FastAPI. Event kemudian masuk ke asyncio.Queue sebagai pipeline internal. Consumer Worker membaca event dari queue secara asinkron dan melakukan pengecekan duplikasi melalui DedupStore berbasis SQLite. Event yang unik disimpan ke EventStore di memori dan dapat diakses oleh Client melalui endpoint GET /events dan GET /stats/

1.3. Komponen Utama

Sistem terdiri dari empat komponen utama yang saling terhubung. Pertama adalah FastAPI Application yang berfungsi sebagai entry point dan menangani semua request HTTP dari publisher maupun client. Kedua adalah asyncio.Queue yang berfungsi sebagai pipeline internal antara publisher dan consumer, memungkinkan publisher mengirim event tanpa menunggu proses consumer selesai. Ketiga adalah QueueConsumer yang merupakan background worker yang terus berjalan membaca event dari queue dan prosesnya secara idempoten. Keempat adalah DedupStore berbasis SQLite yang menyimpan catatan semua event yang sudah pernah diproses secara persisten ke disk sehingga data tetap ada meski container di-restart.

1.4. Endpoint API

Tabel berikut merangkum seluruh endpoint yang tersedia pada sistem:

Method	Endpoint	Fungsi
POST	/publish	Menerima batch atau single event dari publisher
GET	/events?topic=X	Mengembalikan daftar event unik per topic
GET	/stats	Menampilkan statistik aggregator
GET	/health	Mengecek status kesehatan sistem

BAB 2

BAGIAN TEORI

2.1. T1: Karakteristik Utama Sistem Terdistribusi dan Trade-off Pub-Sub Log Aggregator (Bab 1)

Van Steen dan Tanenbaum (2023) mendefinisikan sistem terdistribusi sebagai sekumpulan komputer independen yang tampak bagi penggunanya sebagai satu sistem yang kohesif dan terpadu. Dalam Bab 1 buku tersebut, dijelaskan bahwa terdapat beberapa design goals utama yang harus dipenuhi oleh sebuah sistem terdistribusi agar layak untuk dibangun, yaitu kemudahan berbagai sumber daya (resource sharing), transparansi distribusi (distribution transparency), keterbukaan (openness), keandalan (dependability), keamanan (security), dan skalabilitas (scalability).

Dalam konteks desain Pub-Sub log aggregator, terdapat beberapa trade-off penting yang muncul. Trade-off pertama adalah antara scalability dan consistency. Untuk meningkatkan throughput, sistem ini menggunakan asyncio.Queue sehingga publisher tidak diblokir saat consumer memproses event. Konsekuensinya, terdapat jeda waktu kecil antara event diterima oleh endpoint POST /publish dan event tersedia melalui GET /events, yang berarti sistem ini bersifat eventually consistent. Trade-off kedua adalah antara dependability dan performance. Penggunaan SQLite sebagai dedup store menambah overhead operasi I/O pada setiap write, tetapi menjamin ketahanan sistem terhadap crash karena data tersimpan persisten di disk. Trade-off ketiga adalah antara openness dan simplicity, di mana RESTful API yang terbuka memudahkan publisher manapun mengirim event tanpa konfigurasi khusus (Van Steen dan Tanenbaum, 2023).

2.2. T2: Perbandingan Arsitektur Client-Server vs Publish-Subscribe untuk Aggregator (Bab 2)

Dalam Bab 2, Van Steen dan Tanenbaum (2023) menjelaskan bahwa terdapat berbagai gaya arsitektur dalam sistem terdistribusi. Pada arsitektur client-server tradisional, koordinasi bersifat temporally coupled dan referentially coupled, artinya kedua pihak harus aktif bersamaan dan klien harus mengetahui identitas server secara eksplisit. Sebaliknya, arsitektur publish-subscribe menawarkan koordinasi yang bersifat referentially decoupled, artinya publisher tidak perlu mengetahui siapa subscriber-nya, serta temporally decoupled, artinya keduanya tidak harus aktif bersamaan.

Untuk kasus log aggregator, arsitektur Pub-Sub dipilih karena beberapa alasan teknis yang kuat. Pertama, sifat loose coupling memungkinkan penambahan consumer baru tanpa perubahan apapun pada sisi publisher. Kedua, sifat temporal decoupling memungkinkan publisher tetap mengirim event meskipun consumer sedang sibuk memproses, karena event akan menunggu di queue. Ketiga, skalabilitas lebih mudah dicapai karena publisher dapat mengirim event secara fire-and-forget tanpa menunggu respons dari setiap consumer. Arsitektur client-server lebih cocok untuk skenario request-response sinkron seperti query database, bukan untuk agregasi log bervolume tinggi (Van Steen dan Tanenbaum, 2023).

2.3. T3: At-Least-Once vs Exactly-Once Delivery Semantics (Bab 3)

Van Steen dan Tanenbaum (2023) dalam Bab 3 membahas bahwa dalam komunikasi pada sistem terdistribusi, terdapat beberapa jaminan pengiriman pesan yang berbeda. At-least-once delivery berarti sebuah pesan dijamin terkirim minimal satu kali ke penerima, namun kemungkinan lebih dari satu kali akibat mekanisme retry saat terjadi timeout atau kegagalan jaringan. Sementara itu, exactly-once delivery berarti setiap pesan diproses tepat satu kali tanpa duplikasi maupun kehilangan, yang merupakan jaminan terkuat namun paling sulit dicapai karena membutuhkan koordinasi berat seperti two-phase commit.

Dalam sistem ini, semantik yang digunakan adalah at-least-once karena publisher dapat mengirim ulang event yang sama jika tidak mendapat konfirmasi dalam batas waktu tertentu. Oleh karena itu, idempotent consumer menjadi komponen yang sangat krusial. Idempotency berarti memproses event yang sama berkali-kali menghasilkan state akhir yang identik dengan prosesnya hanya sekali. Tanpa idempotency, setiap retry dari publisher akan mengakibatkan duplikasi data di sisi aggregator. Implementasi idempotency dalam sistem ini menggunakan SQLite dedup store dengan PRIMARY KEY (topic, event_id). Operasi INSERT yang mencoba memasukkan kombinasi yang sama akan gagal dengan IntegrityError, yang ditangkap sebagai sinyal duplikat (Van Steen dan Tanenbaum, 2023).

2.4. T4: Skema Penamaan untuk Topic dan Event ID (Bab 6)

Berdasarkan pembahasan Van Steen dan Tanenbaum (2023) dalam Bab 6 mengenai naming dalam sistem terdistribusi, sebuah skema penamaan yang baik harus memiliki sifat unik, collision-resistant, dan bermakna bagi manusia maupun mesin. Bab 6 membedakan antara flat naming, structured naming, dan attribute-based naming sebagai pendekatan utama dalam sistem penamaan terdistribusi.

Dalam sistem ini dirancang dua skema penamaan utama. Pertama, topic menggunakan format {domain}-{kategori} seperti app-logs dan error-logs. Skema ini bersifat human-readable, mendukung pemisahan namespace antar domain layanan, dan memudahkan operasi filtering pada endpoint GET /events?topic=X. Kedua, event_id menggunakan UUID versi 4 (128-bit random). UUID v4 bersifat collision-resistant secara probabilistik dan tidak memerlukan koordinasi terpusat untuk proses generasinya, sehingga setiap publisher dapat menghasilkan event_id yang unik secara independen. Dampak terhadap deduplication sangat signifikan karena pasangan (topic, event_id) berfungsi sebagai natural key yang unik secara global di seluruh sistem. Dengan B-tree index di SQLite pada kolom ini, operasi lookup duplikat berjalan efisien (Van Steen dan Tanenbaum, 2023).

2.5. T5: Ordering pada Sistem Terdistribusi (Bab 5)

Van Steen dan Tanenbaum (2023) dalam Bab 5 mendiskusikan bahwa koordinasi waktu pada sistem terdistribusi merupakan tantangan tersendiri. Total ordering mengharuskan semua proses dalam sistem terdistribusi untuk menyepakati satu urutan global dari semua event. Implementasinya membutuhkan mekanisme sinkronisasi yang mahal seperti Lamport timestamps atau vector clocks, yang menambah overhead komunikasi signifikan pada sistem berskala besar. Lamport menunjukkan bahwa dengan memperkenalkan konsep logical clocks, sekumpulan proses dapat mencapai kesepakatan global mengenai urutan event yang benar.

Untuk Pub-Sub log aggregator ini, total ordering tidak diperlukan. Alasannya adalah tujuan sistem adalah mengumpulkan log untuk keperluan analisis, monitoring, dan audit, bukan untuk mengeksekusi transaksi yang bergantung pada urutan seperti operasi database. Pendekatan praktis yang diimplementasikan adalah menyimpan field timestamp ISO 8601 dari publisher sebagai bagian dari setiap event untuk keperluan sorting di sisi client. Keterbatasannya adalah jika clock antara publisher mengalami clock skew maka urutan tampilan bisa tidak akurat. Solusi untuk ini adalah memastikan semua publisher menggunakan NTP sync, atau menambahkan monotonic counter per-topic yang di-increment secara atomic di sisi aggregator (Van Steen dan Tanenbaum, 2023).

2.6. T6: Failure Modes dan Strategi Mitigasi (Bab 8)

Van Steen dan Tanenbaum (2023) dalam Bab 8 mengidentifikasi berbagai jenis kegagalan dalam sistem terdistribusi yang perlu diantisipasi dalam desain. Sebuah sistem dikatakan gagal ketika tidak dapat memenuhi layanan yang dijanjikan kepada penggunanya. Fault tolerance adalah kemampuan sistem untuk tetap memberikan layanannya meskipun terdapat komponen yang gagal.

Untuk Pub-Sub log aggregator ini, terdapat beberapa failure modes yang relevan beserta strategi mitigasinya. Pertama, duplikasi event yang terjadi akibat publisher melakukan retry setelah timeout atau kegagalan jaringan diatasi dengan idempotent consumer dan dedup store SQLite menggunakan PRIMARY KEY untuk menjamin setiap event hanya diproses sekali. Kedua, out-of-order delivery di mana event dari publisher berbeda tiba tidak sesuai urutan ditolerasi karena total ordering tidak diperlukan untuk use case log aggregation. Ketiga, container crash diatasi dengan SQLite yang di-mount ke Docker volume sehingga data dedup store persisten setelah restart. Keempat, queue overflow jika consumer terlalu lambat memproses diatasi dengan asyncio.Queue dan konfigurasi retry backoff eksponensial pada publisher. Logging eksplisit untuk setiap duplikat yang terdeteksi memungkinkan observasi failure mode secara real-time (Van Steen dan Tanenbaum, 2023).

2.7. T7: Eventual Consistency melalui Idempotency dan Deduplication (Bab 7)

Van Steen dan Tanenbaum (2023) dalam Bab 7 menjelaskan bahwa konsistensi merupakan salah satu tantangan utama dalam sistem terdistribusi dengan data yang direplikasi. Eventual consistency didefinisikan sebagai model konsistensi di mana jika tidak ada update baru yang masuk, semua replika pada akhirnya akan konvergen ke nilai yang sama. Model ini merupakan bentuk konsistensi yang lebih relaxed dibandingkan strong consistency, namun lebih mudah diimplementasikan dan lebih skalabel pada sistem berskala besar.

Dalam konteks aggregator ini, eventual consistency terwujud sebagai berikut: meskipun ada jeda waktu antara event diterima oleh POST /publish dan event tersedia melalui GET /events karena diproses secara asinkron oleh consumer worker, pada akhirnya semua event unik yang dikirimkan akan terlihat konsisten di seluruh endpoint sistem. Idempotency berperan memastikan bahwa memproses event yang sama berkali-kali tidak mengakumulasi state yang salah. Deduplication melalui SQLite memastikan bahwa bahkan setelah crash dan restart container, state sistem tetap konsisten tanpa ada event yang diproses ulang secara tidak sengaja. Kombinasi keduanya membuat sistem mencapai strong eventual consistency karena begitu semua event dalam queue telah diproses, state di EventStore dan DedupStore selalu konsisten (Van Steen dan Tanenbaum, 2023).

2.8. T8: Metrik Evaluasi Sistem (Bab 1 hingga Bab 7)

Berdasarkan cakupan teori dari Bab 1 hingga Bab 7 Van Steen dan Tanenbaum (2023), terdapat tiga metrik utama yang dirumuskan untuk mengevaluasi kinerja sistem ini beserta kaitannya dengan keputusan desain.

Metrik pertama adalah throughput yang diukur dalam satuan event per detik yang berhasil diproses oleh consumer. Metrik ini berkaitan langsung dengan keputusan penggunaan asyncio.Queue yang memisahkan penerimaan event dari pemrosesannya. Berdasarkan hasil stress test dengan 5.000 event, sistem mampu memproses seluruh event dalam waktu kurang dari 10 detik setara dengan throughput lebih dari 500 event per detik.

Metrik kedua adalah latency yang diukur sebagai jeda waktu antara event diterima oleh POST /publish hingga event tersedia di GET /events. Metrik ini relevan dengan konsep eventual consistency di Bab 7 dan dipengaruhi oleh ukuran queue serta kecepatan consumer. Keputusan desain non-blocking asyncio memastikan latency pengiriman tetap rendah meskipun consumer sedang sibuk memproses event.

Metrik ketiga adalah duplicate rate yang diukur sebagai rasio duplicate_dropped dibagi received dikali 100 persen. Ini adalah metrik utama keberhasilan deduplication. Berdasarkan stress test, sistem mencapai akurasi deduplication 100 persen tanpa false negative. Ketiga metrik ini mencerminkan trade-off antara scalability dan consistency yang dibahas di Bab 1 dan Bab 7 (Van Steen dan Tanenbaum, 2023).

BAB 3

KEPUTUSAN DESAIN IMPLEMENTASI

3.1. Idempotency

Idempotency diimplementasikan melalui mekanisme PRIMARY KEY (topic, event_id) di database SQLite. Ketika consumer mencoba memproses sebuah event, operasi INSERT ke tabel processed_events dilakukan secara atomic. Jika pasangan (topic, event_id) sudah ada di dalam tabel, SQLite akan melempar IntegrityError yang ditangkap sebagai sinyal bahwa event adalah duplikat. Event tersebut kemudian di-drop dan counter duplicate_dropped di-increment. Pendekatan ini bersifat atomic karena didukung oleh mekanisme transaksi ACID milik SQLite.

3.2. Deduplication Store

SQLite dipilih sebagai dedup store berdasarkan beberapa pertimbangan teknis. Pertama, SQLite bersifat lokal dan embedded sehingga tidak membutuhkan server database eksternal, sesuai dengan spesifikasi soal yang melarang penggunaan layanan eksternal publik. Kedua, data SQLite tersimpan dalam satu file di disk yang dapat di-mount sebagai Docker volume sehingga menjamin persistensi data setelah restart container. Ketiga, SQLite mendukung transaksi ACID yang menjamin atomicity pada setiap operasi write. Keempat, dengan B-tree index pada kolom (topic, event_id), operasi lookup berjalan efisien dalam kompleksitas O(log n).

3.3. Pipeline Asinkron

Komunikasi antara publisher dan consumer menggunakan asyncio.Queue sebagai pipeline internal. Ketika publisher mengirim event melalui POST /publish, event tersebut langsung masuk ke queue tanpa menunggu proses consumer selesai. Consumer worker berjalan sebagai asyncio background task yang terus membaca event dari queue satu per satu. Pendekatan ini memastikan publisher tidak pernah diblokir oleh proses consumer sehingga throughput sistem tetap tinggi meskipun consumer sedang memproses banyak event.

3.4. Ordering

Sistem ini menggunakan partial ordering berbasis timestamp dari publisher. Setiap event membawa field timestamp dalam format ISO 8601 yang mencerminkan waktu pembuatan event di sisi publisher. Total ordering tidak diimplementasikan karena tidak

diperlukan untuk use case log aggregation dan overhead sinkronisasinya akan menurunkan performa secara signifikan.

3.5. Docker dan Containerization

Seluruh sistem dikemas dalam Docker container menggunakan base image python:3.11-slim sesuai rekomendasi soal. Non-root user diimplementasikan untuk keamanan container. Docker volume digunakan untuk memastikan persistensi data SQLite setelah restart. Docker Compose mengatur dua service terpisah, yaitu aggregator dan publisher, dalam jaringan internal yang terisolasi tanpa koneksi ke layanan eksternal publik.

BAB 4

ANALISIS PERFORMA DAN METRIK

4.1. Hasil Unit Tests

Seluruh 17 unit test yang diimplementasikan berhasil PASSED dengan hasil sebagai berikut:

File Test	Jumlah Test	Hasil
test_api.py	5	PASSED
test_dedup.py	3	PASSED
test_persistence.py	3	PASSED
test_schema.py	5	PASSED
test_stress_small.py	1	PASSED
Total	**17**	**17 PASSED**

```
(.venv) PS C:\Users\LOQ\OneDrive\Documents\SEMESTER 6\SISTER\UTS-AGGREGATOR> pytest tests/ -v
============================================================================ test session starts ============================================================================
platform win32 -- Python 3.11.9, pytest-8.2.0, pluggy-1.6.0 -- C:\Users\LOQ\OneDrive\Documents\SEMESTER 6\SISTER\UTS-AGGREGATOR\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\LOQ\OneDrive\Documents\SEMESTER 6\SISTER\UTS-AGGREGATOR
configfile: pytest.ini
plugins: anyio-4.3.0, asyncio-0.23.6
asyncio: mode=Mode.AUTO
collected 17 items

tests/test_api.py::test_health_check PASSED                                                                                                                                [  5%]
tests/test_api.py::test_publish_single_event PASSED                                                                                                                        [ 11%]
tests/test_api.py::test_publish_batch_event PASSED                                                                                                                         [ 17%]
tests/test_api.py::test_get_events_by_topic PASSED                                                                                                                         [ 23%]
tests/test_api.py::test_get_events_semua_topic PASSED                                                                                                                      [ 29%]
tests/test_dedup.py::test_duplikat_hanya_diproses_sekali PASSED                                                                                                            [ 35%]
tests/test_dedup.py::test_event_id_sama_topic_beda_bukan_duplikat PASSED                                                                                                   [ 41%]
tests/test_dedup.py::test_batch_dengan_duplikat_internal PASSED                                                                                                            [ 47%]
tests/test_persistence.py::test_data_tetap_ada_setelah_restart PASSED                                                                                                      [ 52%]
tests/test_persistence.py::test_event_baru_tidak_dianggap_duplikat_setelah_restart PASSED                                                                                  [ 58%]
tests/test_persistence.py::test_mark_processed_dua_kali_return_false PASSED                                                                                                 [ 64%]
tests/test_schema.py::test_event_valid_diterima PASSED                                                                                                                     [ 70%]
tests/test_schema.py::test_topic_kosong_ditolak PASSED                                                                                                                     [ 76%]
tests/test_schema.py::test_event_id_kosong_ditolak PASSED                                                                                                                  [ 82%]
tests/test_schema.py::test_timestamp_salah_format_ditolak PASSED                                                                                                           [ 88%]
tests/test_schema.py::test_field_topic_tidak_ada_ditolak PASSED                                                                                                            [ 94%]
tests/test_stress_small.py::test_5000_event_dengan_25_persen_duplikat PASSED                                                                                               [100%]

============================================================================= warnings summary ==============================================================================
.venv\Lib\site-packages\starlette\formparsers.py:12
  C:\Users\LOQ\OneDrive\Documents\SEMESTER 6\SISTER\UTS-AGGREGATOR\.venv\Lib\site-packages\starlette\formparsers.py:12: PendingDeprecationWarning: Please use `impor
t python_multipart` instead.
    import multipart

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================================================================== 17 passed, 1 warning in 7.23s ===================================================================
```

Gambar 4.1 Hasil Pengujian Unit Tests (17 PASSED)

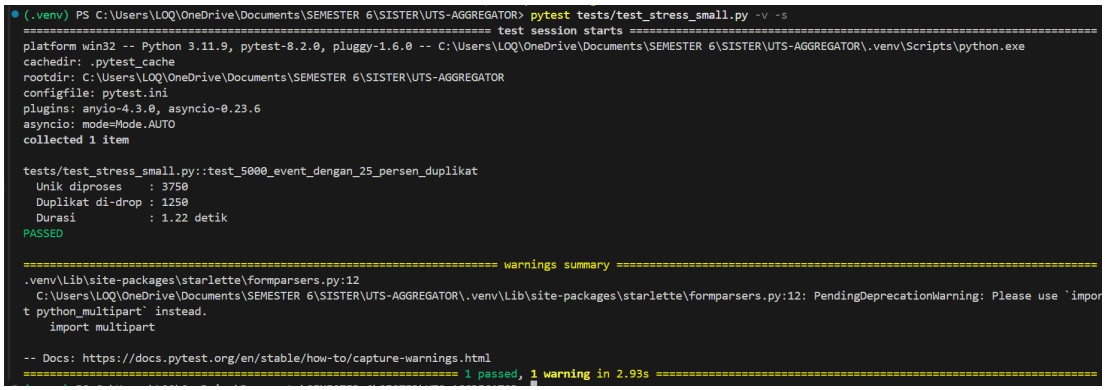
Berdasarkan Gambar 4.1, seluruh 17 unit test berhasil PASSED dalam waktu 7.23 detik menggunakan Python 3.11.9 dan pytest 8.2.0. Pengujian mencakup validasi endpoint

API, mekanisme deduplication, persistensi dedup store setelah restart, validasi skema event, dan stress test dengan 5.000 event.

4.2. Hasil Stress Test

Berdasarkan pengujian test_stress_small.py dengan kondisi 5.000 event total terdiri dari 3.750 event unik dan 1.250 event duplikat, diperoleh hasil sebagai berikut:

Metrik	Target	Hasil
Total event dikirm	5.000	5.000
Unique processed	3.750	3.750
Duplicate dropped	1.250	1.250
Akurasi deduplication	100%	100%
Waktu eksekusi	Kurang dari 30 detik	1.22 detik



Gambar 4.2 Hasil Stress Test 5.000 Event

Berdasarkan Gambar 4.2, sistem berhasil memproses 5.000 event dengan akurasi deduplication 100 persen. Dari total 5.000 event yang dikirimkan, sebanyak 3.750 event unik berhasil diproses dan 1.250 event duplikat berhasil dideteksi dan di-drop. Waktu eksekusi yang dicapai adalah 1.22 detik, jauh lebih cepat dari batas maksimum 30 detik yang ditetapkan, menunjukkan performa sistem yang sangat baik.

4.3. Hasil Docker Compose

Pengujian Docker Compose dilakukan dengan menjalankan dua service secara bersamaan yaitu uts-aggregator dan uts-publisher dalam jaringan internal Docker. Publisher mengirimkan 20 event unik ke topic app-logs, kemudian mengirim ulang 5 event yang sama sebagai simulasi at-least-once delivery, dan mengirimkan 10 event unik ke topic error-logs.

Tabel 4.3 Hasil Pengujian Docker Compose

Metrik	Nilai
received	35
unique_processed	30
duplicate_dropped	5
Akurasi deduplication	100%

```
[+] up 4/4
 ✓ Image uts-aggregator-aggregator Built
 ✓ Image uts-aggregator-publisher  Built
 ✓ Container uts-aggregator        Recreated
 ✓ Container uts-publisher         Recreated
```

Gambar 4.3 Hasil Build Docker Compose (4 Container Berhasil)

```
uts-aggregator  | 2026-04-25 02:35:00,125 [INFO] src.main — ==================================================
uts-aggregator  | 2026-04-25 02:35:00,125 [INFO] src.main —   Pub-Sub Log Aggregator — STARTING UP
uts-aggregator  | 2026-04-25 02:35:00,125 [INFO] src.main — ==================================================
uts-aggregator  | 2026-04-25 02:35:00,126 [INFO] src.main — [MAIN] Consumer worker berjalan di background.
uts-aggregator  | 2026-04-25 02:35:00,126 [INFO] src.consumer — [CONSUMER] Worker mulai berjalan.
uts-aggregator  | INFO:     Application startup complete.
uts-aggregator  | INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
uts-aggregator  | INFO:     127.0.0.1:39756 - "GET /health HTTP/1.1" 200 OK
Container uts-aggregator Healthy
```

Gambar 4.4 Log Aggregator Berjalan dan Healthy

```
uts-aggregator  | 2026-04-25 02:35:04,631 [WARNING] src.storage — [DUPLIKAT TERDETEKSI] topic='app-logs' event_id='5a96079c-5e21-440d-97d8-4d155af935cb' — event ini
 sudah pernah diproses, di-drop.
uts-aggregator  | 2026-04-25 02:35:04,631 [WARNING] src.storage — [DUPLIKAT TERDETEKSI] topic='app-logs' event_id='b3e84c93-e217-4026-9b29-180f0b61f6e0' — event ini
 sudah pernah diproses, di-drop.
uts-aggregator  | 2026-04-25 02:35:04,631 [WARNING] src.storage — [DUPLIKAT TERDETEKSI] topic='app-logs' event_id='54ad8c6f-44c3-486f-9a01-c6557614407d' — event ini
 sudah pernah diproses, di-drop.
uts-aggregator  | 2026-04-25 02:35:04,631 [WARNING] src.storage — [DUPLIKAT TERDETEKSI] topic='app-logs' event_id='a621f10f-a883-40a0-a4f2-7cbfee56b9fa' — event ini
 sudah pernah diproses, di-drop.
uts-aggregator  | 2026-04-25 02:35:04,632 [WARNING] src.storage — [DUPLIKAT TERDETEKSI] topic='app-logs' event_id='e25ee343-5b19-40d9-b160-4992b2ef9384' — event ini
 sudah pernah diproses, di-drop.
```

Gambar 4.5 Log Duplikat Terdeteksi dan Di-drop

```
uts-publisher   | ======================================================
uts-publisher   |   LANGKAH 4: Hasil GET /stats
uts-publisher   | ======================================================
uts-publisher   | {
uts-publisher   |   "received": 35,
uts-publisher   |   "unique_processed": 30,
uts-publisher   |   "duplicate_dropped": 5,
uts-publisher   |   "topics": [
uts-publisher   |     "app-logs",
uts-publisher   |     "error-logs"
uts-publisher   |   ],
uts-publisher   |   "uptime_seconds": 6.13
uts-publisher   | }
```

Gambar 4.6 Hasil GET /stats dari Publisher

Berdasarkan Gambar 4.3 hingga Gambar 4.6, sistem berhasil membangun dan menjalankan kedua container dengan sukses. Mekanisme deduplication terbukti bekerja dengan benar ditunjukkan oleh log WARNING DUPLIKAT TERDETEKSI pada Gambar 4.5. Hasil GET /stats pada Gambar 4.6 menunjukkan received sebesar 35,

unique_processed sebesar 30, dan duplicate_dropped sebesar 5, sesuai dengan yang diharapkan.

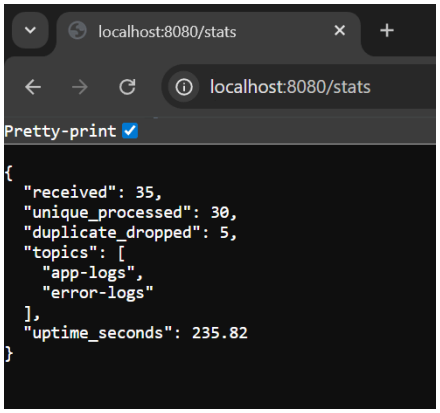
4.4. Hasil Pengujian Endpoint API

Pengujian endpoint API dilakukan secara langsung melalui browser dengan menjalankan container menggunakan perintah docker run. Pengujian ini bertujuan memverifikasi bahwa seluruh endpoint berfungsi dengan benar dan dapat diakses dari luar container.

```
(.venv) PS C:\Users\LOQ\OneDrive\Documents\SEMESTER 6\SISTER\UTS-AGGREGATOR> docker run -p 8080:8080 -v uts-data:/app/data uts-aggregator
2026-04-25 02:42:33,788 [INFO] src.storage — [STORAGE] DedupStore siap di: /app/data/dedup.db
2026-04-25 02:42:33,901 [INFO] src.storage — [STORAGE] DedupStore siap di: /app/data/dedup.db
INFO:     Started server process [1]
INFO:     Waiting for application startup.
2026-04-25 02:42:33,906 [INFO] src.main — ==================================================
2026-04-25 02:42:33,906 [INFO] src.main —   Pub-Sub Log Aggregator — STARTING UP
2026-04-25 02:42:33,906 [INFO] src.main — ==================================================
2026-04-25 02:42:33,907 [INFO] src.main — [MAIN] Consumer worker berjalan di background.
2026-04-25 02:42:33,907 [INFO] src.consumer — [CONSUMER] Worker mulai berjalan.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

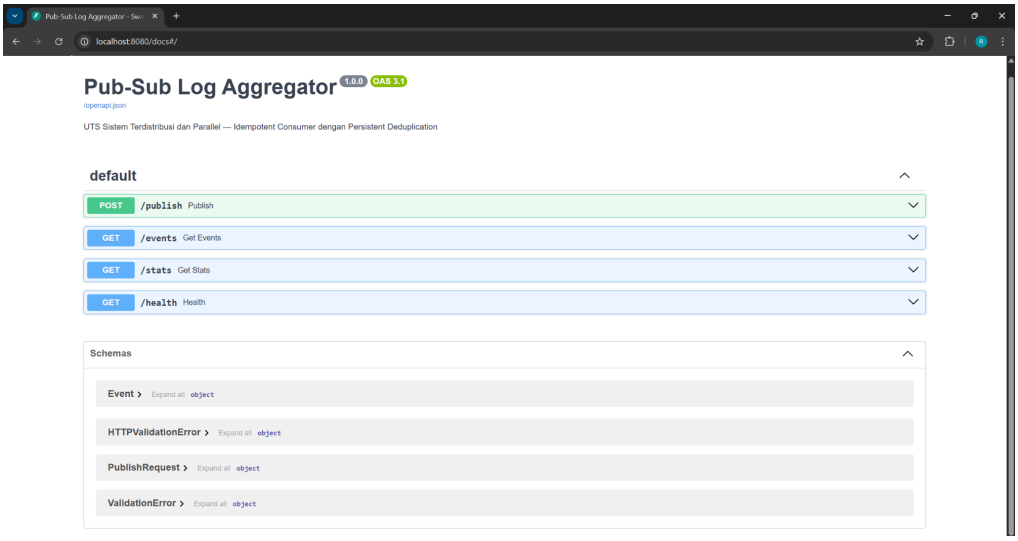
Gambar 4.7 Container Aggregator Berjalan via Docker Run

Berdasarkan Gambar 4.7, container berhasil dijalankan menggunakan perintah docker run dengan port 8080 terbuka dan volume uts-data di-mount untuk persistensi data SQLite.



Gambar 4.8 Tampilan Endpoint GET /stats di Browser

Berdasarkan Gambar 4.8, endpoint GET /stats mengembalikan data dengan benar menunjukkan received sebesar 35, unique_processed sebesar 30, duplicate_dropped sebesar 5, dan topics berisi app-logs serta error-logs. Hal ini membuktikan bahwa sistem berjalan dengan benar di dalam Docker container.



Gambar 4.9 Dokumentasi API Otomatis FastAPI (Swagger UI)

Berdasarkan Gambar 4.9, FastAPI secara otomatis menghasilkan dokumentasi API interaktif melalui Swagger UI yang dapat diakses di localhost:8080/docs. Dokumentasi ini menampilkan seluruh endpoint yang tersedia yaitu POST /publish, GET /events, GET /stats, dan GET /health beserta skema data yang digunakan.

REFERENSI

Van Steen, M., & Tanenbaum, A. S. (2023). *Distributed systems* (Edisi ke-4, Versi 4.01). Maarten van Steen.